

Git & GitHub — Cheat Sheet

Git 2.55.x · GitHub CLI 2.101.x · IntelliJ IDEA 2026.2 · VS Code (current stable) · github.com · default branch **main**

THE THREE AREAS

```
Working Tree --add-->
Staging Area --commit--
> Repository (local) --
push/pull--> Remote
(origin)
<--restore-- <--
reset/checkout--
```

Untracked / unmodified / modified / staged is a file's lifecycle. `git status` shows where every file sits right now. `.git/` holds the whole history; the working tree is just the current checkout of one commit.

SETUP & CONFIG

```
git config --global
user.name "Name"
git config --global
user.email you@x.com
git config --global
init.defaultBranch main
git config --global
core.editor "code -w"
git config --global
pull.rebase false
git config --list --
show-origin
git help <cmd> # or:
git <cmd> -h
```

Auth: HTTPS + Git Credential Manager, or gh auth login;
SSH: `ssh-keygen -t ed25519`, add **key** on GitHub, `ssh -T git@github.com`. PATs are the HTTPS fallback.

CREATE / CLONE

```
git init
git clone
https://github.com/o/r.
git
git clone
git@github.com:o/r.git
gh repo create name --
public --clone
gh repo clone o/r
gh repo fork o/r --
clone
git remote -v
git remote add origin
<url>
```

.gitignore: patterns per project; `core.excludesfile` for a global ignore; `github/gitignore` has templates.

STAGE & COMMIT

```
git add <file>
# or -A / .
git add -p
# stage by hunk
git commit -m "msg"
git commit -a -m "msg"
# tracked files only
git commit --amend
# before pushing only
```

Good messages: 50-char summary line, blank line, 72-col body, imperative mood ("Add", not "Added"); Closes #123 links a GitHub issue.

INSPECT

```
git status
git log --oneline --
graph --all
git log -p -- <path>
git show <sha>
git diff
# working vs staged
git diff --staged
# staged vs HEAD
```

UNDO

Want to...	Command
Unstage a file	<code>git restore -</code> <code>-staged <f></code>
Discard working changes	<code>git restore</code> <code><f></code>
Fix last commit	<code>git commit --</code> <code>amend</code>
Undo a pushed commit	<code>git revert</code> <code><sha></code>
Rewind, keep changes	<code>git reset --</code> <code>soft <sha></code>
Rewind, unstage	<code>git reset --</code> <code>mixed <sha></code>
Rewind, discard all	<code>git reset --</code> <code>hard <sha></code>

***git reset --hard and git push --force** rewrite/discard history — never on a shared branch; prefer **revert** once pushed.*

BRANCHES

```
git branch
# list local
git branch -a
# + remote-tracking
git switch -c
feature/12
git branch -m old new
git branch -d name
# safe delete
git branch -D name
git branch -u
origin/main
git branch -vv
```

`origin/main` is a remote-tracking branch; `-u` sets the upstream a local branch pushes/pulls against.

CHECKOUT VS SWITCH VS RESTORE

Old (checkout)	New
<code>checkout</code> <code><branch></code>	<code>switch</code> <code><branch></code>
<code>checkout -b</code> <code><n></code>	<code>switch -c</code> <code><n></code>
<code>checkout --</code> <code><f></code>	<code>restore <f></code> <code><f></code>
<code>checkout --</code> <code><sha></code>	<code>restore --</code> <code>source <sha></code> <code><f></code>
<code>git switch --detach</code> <code>v1.2.0</code> → detached HEAD (read-only browsing); <code>switch -c</code> to keep new commits on a real branch. <code>gh pr checkout <n></code> checks out a PR locally.	

STASH

```
git stash #
or: stash push -m "wip"
git stash list
git stash pop #
apply + drop
git stash apply #
apply, keep entry
git switch -m other #
merge stash on switch
```

TAGS & RELEASES

```
git tag
# list
git tag -a v1.4.0 -m
"1.4.0"
git tag -a v1.3.1 <sha>
# tag older commit
git show v1.4.0
git push origin v1.4.0
# tags: not auto-pushed
git push --tags
git push origin --
delete v1.4.0
gh release create
v1.4.0 --generate-notes
```

Lightweight tag = a name; annotated tag = name + message + signature (recommended). Tags can trigger GitHub Actions and tag rulesets.

REMOTES: FETCH / PULL / PUSH

```
git fetch --prune
git pull
# fetch + merge
git pull --rebase
git push
git push -u origin
feature/12
git push --force-with-
lease # never --force

fetch only downloads; pull
= fetch + integrate. A rejected
non-fast-forward push:
pull/rebase, resolve, push again.
push.autoSetupRemote auto-
sets the upstream.
```

MERGE / REBASE / CONFLICTS

```
git merge feature
# fast-fwd or 3-way
git merge --no-ff
feature
git merge --squash
feature
git rebase main
# replay onto main
git rebase --continue
git rebase --abort
git merge --abort
```

Conflict markers: <<<<<< /
===== / >>>>>>. Edit, then
`git add <f>` + `git commit`
(or `merge --continue`).

Golden rule: never rebase commits already pushed/shared with others.

GITHUB FLOW — SIX STEPS

- 1 Branch off `main`: `git switch -c feature/12`.
- 2 Commit small, focused changes.
- 3 Push: `git push -u origin feature/12`.
- 4 Open a pull request early (draft if unfinished).
- 5 Review, discuss, push fixes; checks run.
- 6 Merge on GitHub, delete the branch, pull `main` locally.

PULL REQUESTS

```
gh pr create --base
main --head feature/12 \
--title "..." --body
"..." --draft
gh pr list
gh pr view 42 --web
gh pr checkout 42
gh pr review 42 --
approve
gh pr merge 42 --squash
--delete-branch
```

Merge method	Result
Merge commit	keeps every commit + a merge commit
Squash and merge	all commits → one commit
Rebase and merge	commits replayed, no merge commit

Closes #123 in the description links/auto-closes an issue. PRs from forks target the upstream repo.

IDE STRIP

IntelliJ IDEA — Commit: `Ctrl+K` / `⌘K`; Push: `Ctrl+Shift+K` / `⌘⌘K`; Update Project: `Ctrl+T` / `⌘T`; Git log: `Alt+9` / `⌘9`; branches popup on the status bar; **Git > GitHub > Create Pull Request**; Shelve vs Stash for uncommitted work.
VS Code — Source Control view: `Ctrl+Shift+G` / `⌘ⓖ`; Command Palette: `Ctrl+Shift+P` / `⌘ⓅP`; **Git: Clone, Git: Checkout to..., Sync Changes, Git: Stash / Pop Stash**; *GitHub Pull Requests and Issues* extension — **Create Pull Request**, PR review mode, status-bar sync indicator.